

V2 of AI Implementation - Configuration Module

Document purpose

Knowledge Base Architecture Changes

Previous Approach (RAG-Only):

New Hybrid Approach

Code-Generated Documentation

Function Calling Architecture

Overview

Sequence Diagram

Component Diagram

The `get_live_parameter_value` Tool

Purpose

Function Definition

Response Structure

Query Status Codes

Implementation Classes

Class Diagram

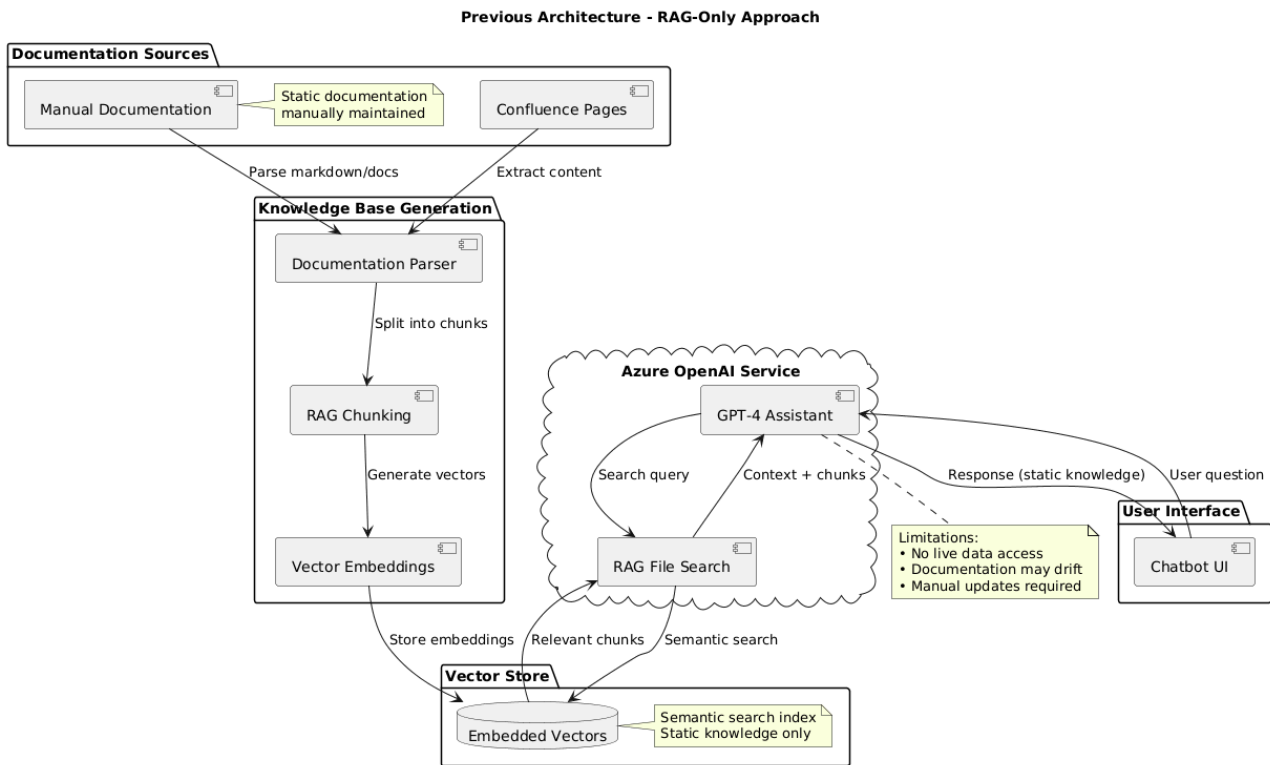
Document purpose

This document describes the enhancements made to the AI Assistant implementation in the Configuration Module since the last documentation (December 10, 2025). The key changes include:

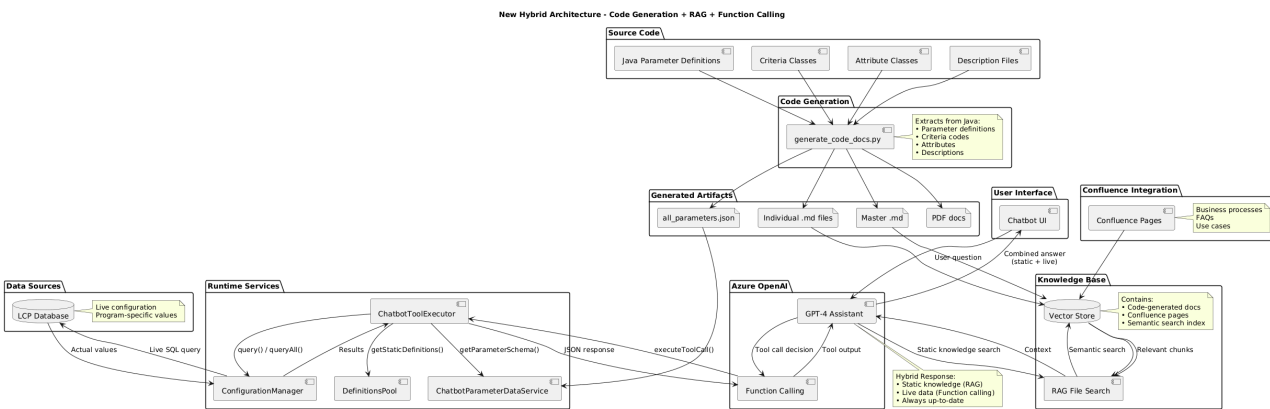
1. **Code-Generated Knowledge Base** - Parameter documentation now extracted directly from source code
2. **OpenAI Function Calling** - Real-time tool execution for live configuration queries
3. `get_live_parameter_value` **Tool** - New capability to query actual program configurations

Knowledge Base Architecture Changes

Previous Approach (RAG-Only):



New Hybrid Approach



Code-Generated Documentation

The `generate_code_docs.py` script extracts parameter definitions directly from Java source code:

Input Sources:

- `config-module/definitions/src/main/java/.../definitions/*.java` - Parameter enums
- `*Criteria.java` - Filter criteria definitions
- `*Attributes.java` - Configurable attributes
- `*Descriptions.java` - Human-readable descriptions

Output Formats:

Format	Purpose	Location
<code>all_parameters.json</code>	Structured schema for tools	<code>chatbot/all_parameters.json</code>
<code>*.md</code> (individual)	Per-parameter RAG chunks	<code>code_generated_docs/parameters/</code>
<code>master.md</code>	Complete reference	<code>code_generated_docs/</code>
<code>*.pdf</code>	Human-readable docs	<code>code_generated_docs/</code>
<code>*.jsonl</code>	Embedding generation	<code>code_generated_docs/</code>

Data Structure per Parameter:

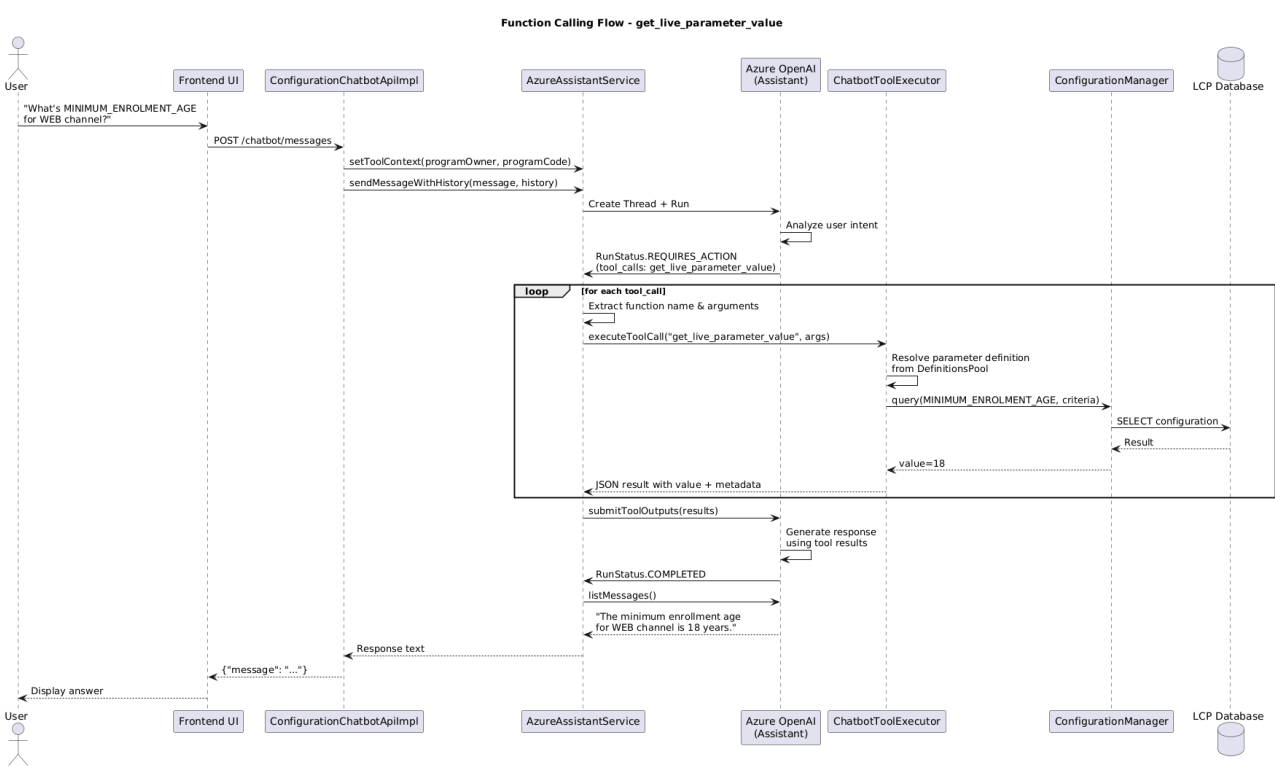
```
1  {
2    "short_name": "PARTNER_LOYALTY_CURRENCIES",
3    "category": "Accrual",
4    "description": "Defines loyalty currencies for partner accruals",
5    "data_type": "Map",
6    "security_group": "CLIENT_VISIBLE",
7    "deprecated": false,
8    "criteria": [
9      {"code": "partnerCode", "required": true},
10     {"code": "action", "required": true},
11     {"code": "currencyCode", "required": true}
12   ],
13   "attributes": [
14     {"code": "validityStartDate", "required": false},
15     {"code": "validityEndDate", "required": false}
16   ]
17 }
```

Function Calling Architecture

Overview

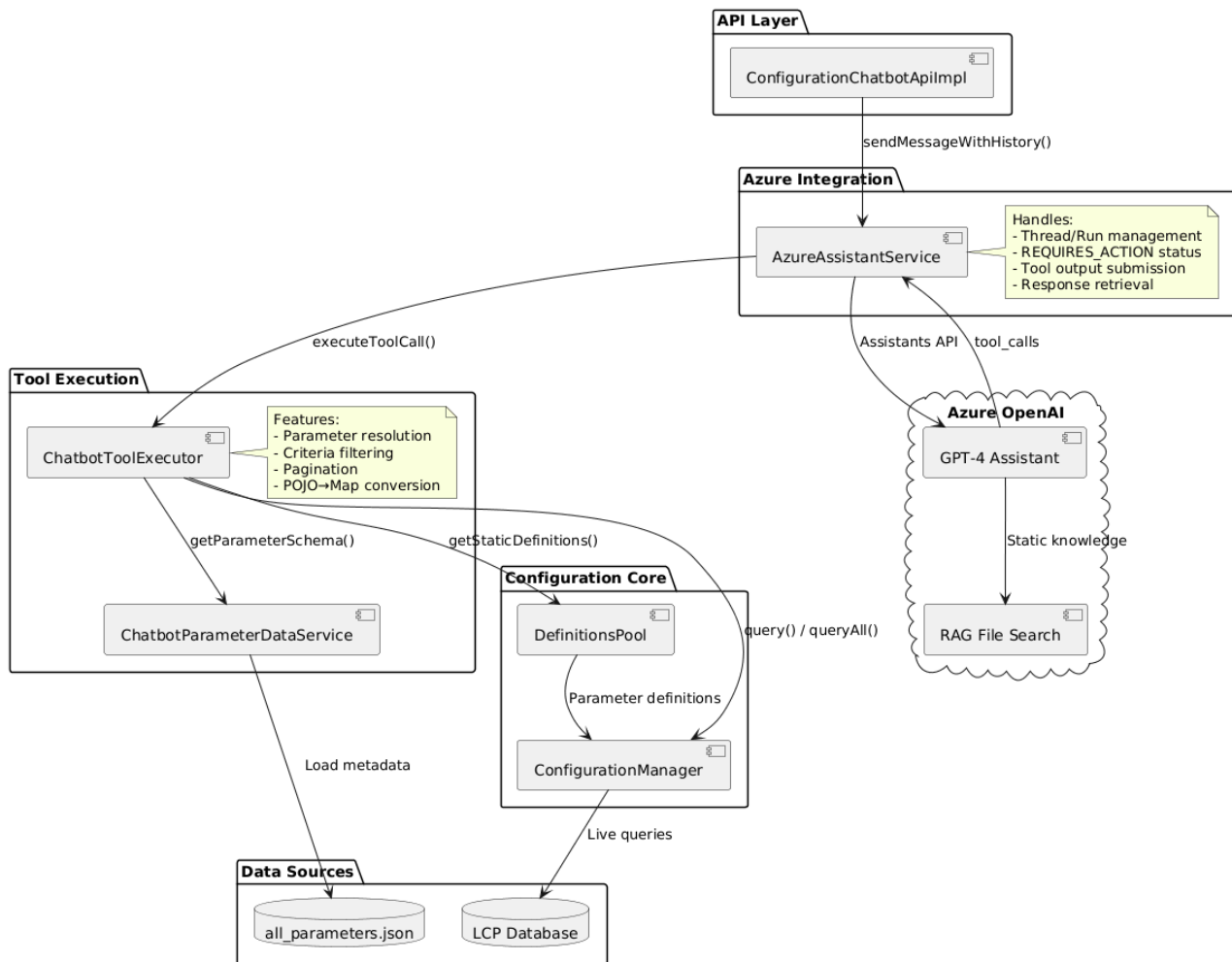
OpenAI Function Calling enables the AI Assistant to execute real-time queries against the live configuration database, providing actual values instead of just documentation.

Sequence Diagram



Component Diagram

Chatbot Components - Function Calling Architecture



The `get_live_parameter_value` Tool

Purpose

Enables the AI Assistant to query **actual configuration values** from the each program's database.

Function Definition

```
1 {
2   "name": "get_live_parameter_value",
3   "description": "Get live config values. RULES: 1) ALWAYS set list_all=true when filtering or listing. 2) Extract criteria from user input: 'for 1A'→partnerCode:1A, 'WEB channel'→channel:WEB. 3) Use UPPERCASE values.",
4   "parameters": {
5     "type": "object",
6     "properties": {
7       "parameter_name": {
8         "type": "string",
9         "description": "Parameter name in UPPERCASE (e.g., PARTNER_LOYALTY_CURRENCIES)"
```

```

10     },
11     "list_all": {
12         "type": "boolean",
13         "description": "Set TRUE when listing/filtering configurations"
14     },
15     "criteria": {
16         "type": "object",
17         "description": "Filter key-value pairs (partnerCode, action, channel, etc.)",
18         "additionalProperties": { "type": "string" }
19     },
20     "limit": {
21         "type": "integer",
22         "description": "Max results (default 15, max 100)"
23     },
24     "offset": {
25         "type": "integer",
26         "description": "Skip N results for pagination"
27     }
28 },
29 "required": ["parameter_name"]
30 }
31 }

```

Response Structure

```

1  {
2      "parameter_name": "MINIMUM_ENROLMENT_AGE",
3      "program": "KAL/KAL",
4      "query_type": "filtered",
5      "query_status": "success",
6      "metadata": {
7          "category": "Membership",
8          "data_type": "Integer",
9          "security_group": "CLIENT_VISIBLE",
10         "deprecated": false,
11         "available_criteria": [
12             {"code": "channel", "required": true},
13             {"code": "subType", "required": false}
14         ]
15     },
16     "criteria_used": {
17         "channel": "WEB"
18     },
19     "values": [
20         {
21             "criteria": {"channel": "WEB"},
22             "data": {"value": 18}
23         }
24     ],
25     "pagination": {
26         "total_count": 1,
27         "displayed_count": 1,
28         "offset": 0,
29         "limit": 15,
30         "has_more": false
31     }
32 }

```

Query Status Codes

Status	Meaning
success	Value(s) found and returned

<code>not_configured</code>	Parameter has no value set
<code>entry_exists_empty</code>	Entry exists but attributes are empty
<code>requires_criteria</code>	Parameter needs criteria (use <code>list_all=true</code>)
<code>filtered_no_matches</code>	No items match the provided criteria
<code>no_matches</code>	No configurations exist for this parameter
<code>error</code>	Query failed (see error message)

Implementation Classes

Class Diagram

1

Chatbot Service Classes

